

Dataset Annotation

Dataset Annotation

1. Dataset Collection
 - 1.1 Image Extraction
 - 1.2 Dataset Path
 2. Label Studio Usage
 - 2.1 Label Studio Installation
 - 2.2 Label Studio Startup
 - 2.2.1 Shared Folder Permissions
 - 2.2.2 Start Label Studio
 - 2.2.3 Access Label Studio
 - Host Machine
 - Same Local Network
 - First-time Use
 3. Dataset Annotation
 - 3.1 Create Project
 - 3.2 Project Name
 - 3.3 Import Training Set
 - 3.4 Label Settings
 - 3.5 Dataset Annotation
 - 3.6 Export Dataset
 - 3.7 Dataset Directory Structure
 - 3.8 Dataset Configuration File
- References

For recognizing specific objects or improving object recognition accuracy, users need to train models themselves. Dataset collection and annotation are an indispensable part.

1. Dataset Collection

Note: For recognition in specific scenarios, users can use a camera to record videos of objects from various angles, then extract images from the videos as training sets.

1.1 Image Extraction

Provide a simple example: Extract one frame from the video every 15 frames as a dataset

```
import cv2
import os

# Path to the input video file
video_file = '/home/pi/ultralytics26/data/yahboom_data/orange_video/orange.mkv'

# Path to the output folder to save the frames
output_folder = '/home/pi/ultralytics26/data/yahboom_data/orange_pic'

# Check if the output folder exists, if not, create it
if not os.path.exists(output_folder):
    os.makedirs(output_folder)

# Open the video file using OpenCV
```

```

cap = cv2.VideoCapture(video_file)

# Get the total number of frames in the video
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

# Get the frames per second (fps) of the video
fps = cap.get(cv2.CAP_PROP_FPS)

# Frame counter to keep track of the current frame number
frame_number = 0

# Define the interval for extracting frames (every 15th frame)
frame_interval = 15

# Initialize a new counter for saved frames (starting from 1)
saved_frame_number = 1

while True:
    ret, frame = cap.read()

    # If the frame is not successfully read, exit the loop
    if not ret:
        break

    # Save every 15th frame
    if frame_number % frame_interval == 0:
        # Format the frame number starting from 1 (e.g., 1.png, 2.png)
        image_name = f'{saved_frame_number}.png'
        image_path = os.path.join(output_folder, image_name)

        # Save the current frame as a PNG image
        cv2.imwrite(image_path, frame)
        print(f'Saving frame {frame_number}/{total_frames} as {image_name}')

        # Increment the saved frame counter
        saved_frame_number += 1

    # Increment the frame counter for the video
    frame_number += 1

# Release the video capture object
cap.release()

# Print message when processing is complete
print("Video processing complete! Every 15th frame saved as an image!")

```

1.2 Dataset Path

Example video: /home/pi/ultralytics26/data/yahboom_data/orange_video

Example images: /home/pi/ultralytics26/data/yahboom_data/orange_pic

2. Label Studio Usage

Label Studio is an open-source data annotation platform for data annotation and annotation task management, supporting various types of data input and annotation formats.

2.1 Label Studio Installation

```
docker pull heartexlabs/label-studio:latest
```

```
pi@raspberrypi:~$ docker pull heartexlabs/label-studio:latest
latest: Pulling from heartexlabs/label-studio
f626fba1463b: Pull complete
0e3bea94763e: Pull complete
78404026428a: Pull complete
efae213b071: Pull complete
02038e343d20: Pull complete
b0e726620daa: Waiting
93a340e67a61: Download complete
0998eda90d0f: Download complete
007a5d0ec1a6: Waiting
587a2af1ca28: Download complete
ef6ad7fa0ba0: Download complete
aa5bc52c9753: Download complete
537da70a57d8: Pulling fs layer
7acbfff5d0620: Waiting
3be5305e2cf1: Waiting
99102fa96398: Waiting
c935a3a4a6b1: Waiting
b49b07494fa1: Waiting
d4007733b13d: Waiting
8e1d719702c2: Waiting
898f839866eb: Waiting
```

2.2 Label Studio Startup

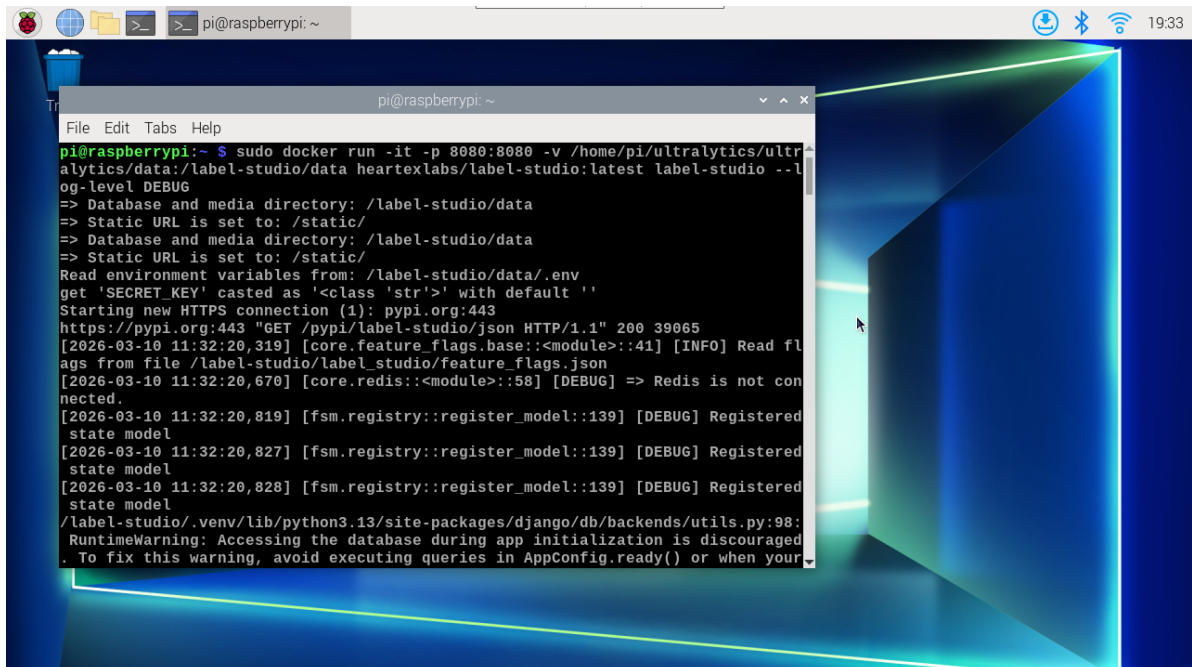
2.2.1 Shared Folder Permissions

Share the prepared dataset to the corresponding folder in Label Studio:

```
sudo chmod 777 -R /home/pi/ultralytics26/data/
```

2.2.2 Start Label Studio

```
sudo docker run -it -p 8080:8080 -v /home/pi/ultralytics/ultralytics/data:/label-studio/data heartexlabs/label-studio:latest label-studio --log-level DEBUG
```

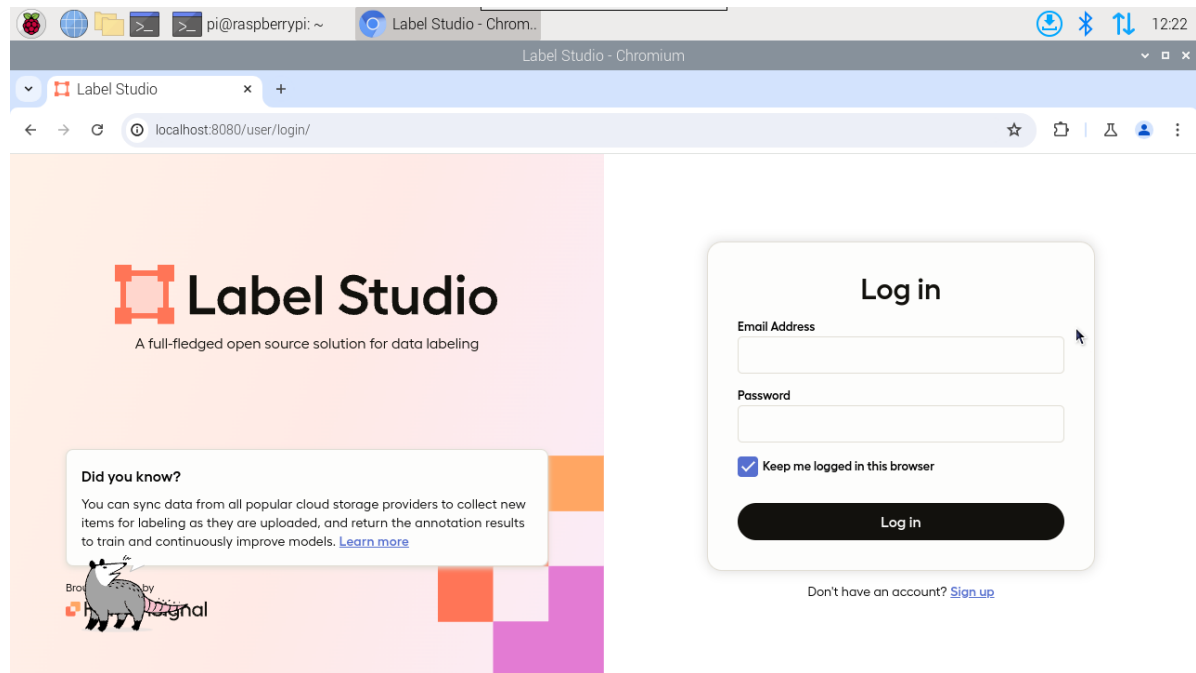


2.2.3 Access Label Studio

Host Machine

Access method for Pi Orin series development boards. First-time access requires entering username and password.

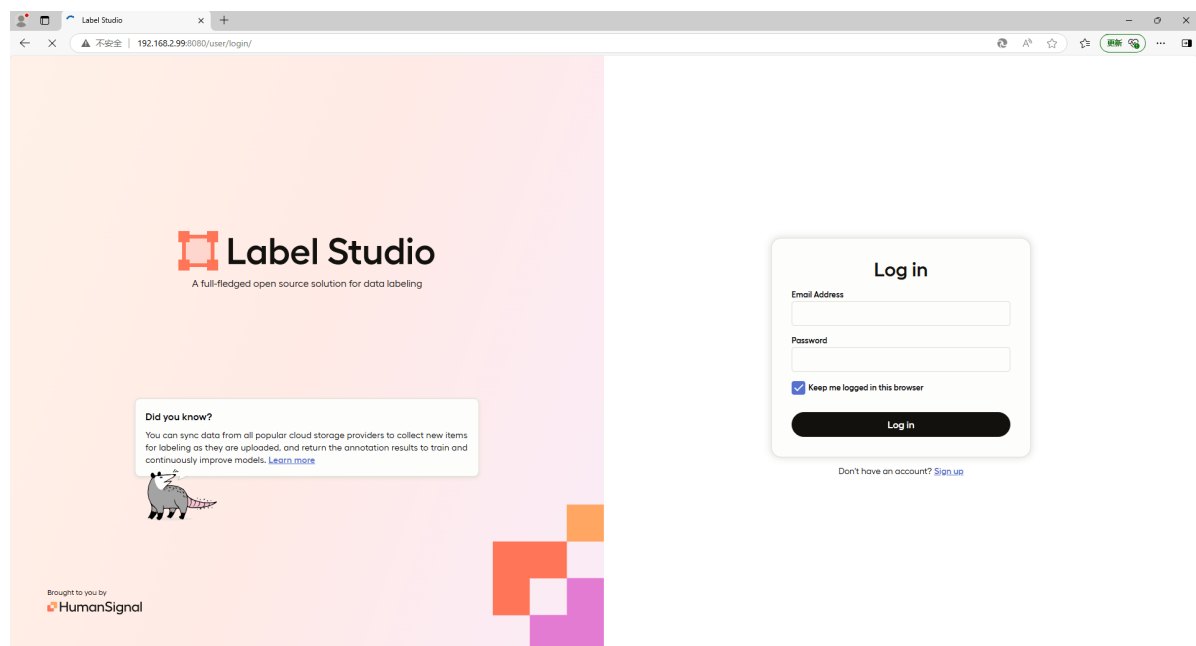
`http://localhost:8080/`



Same Local Network

Devices on the same local network can access via board IP:8080

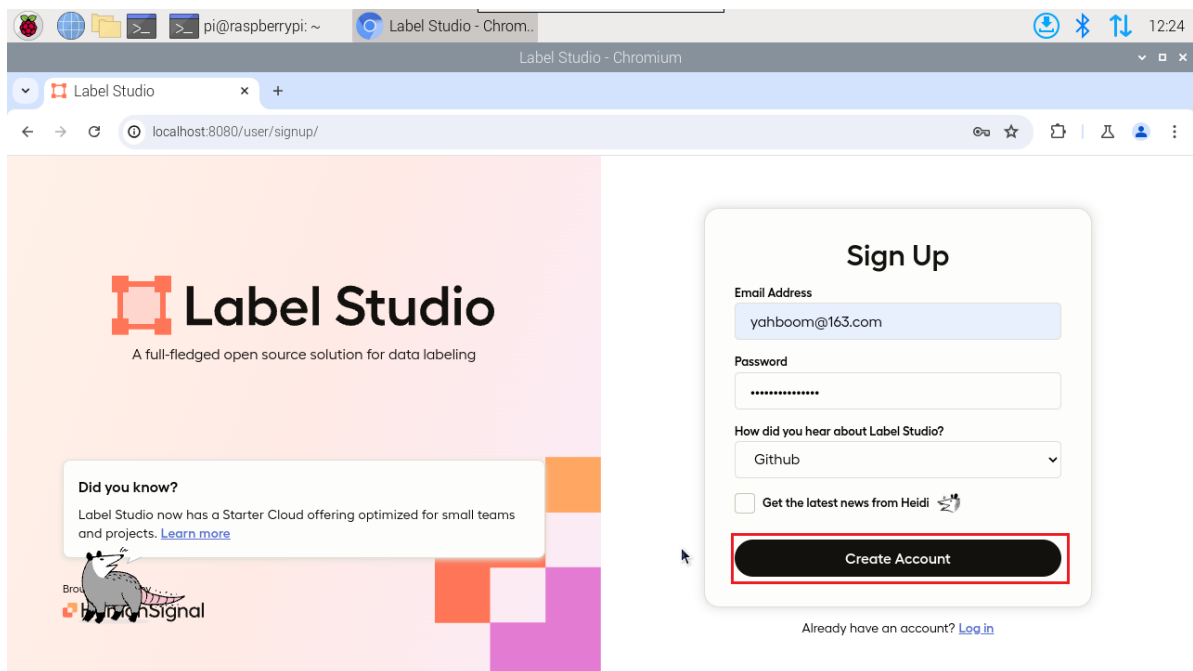
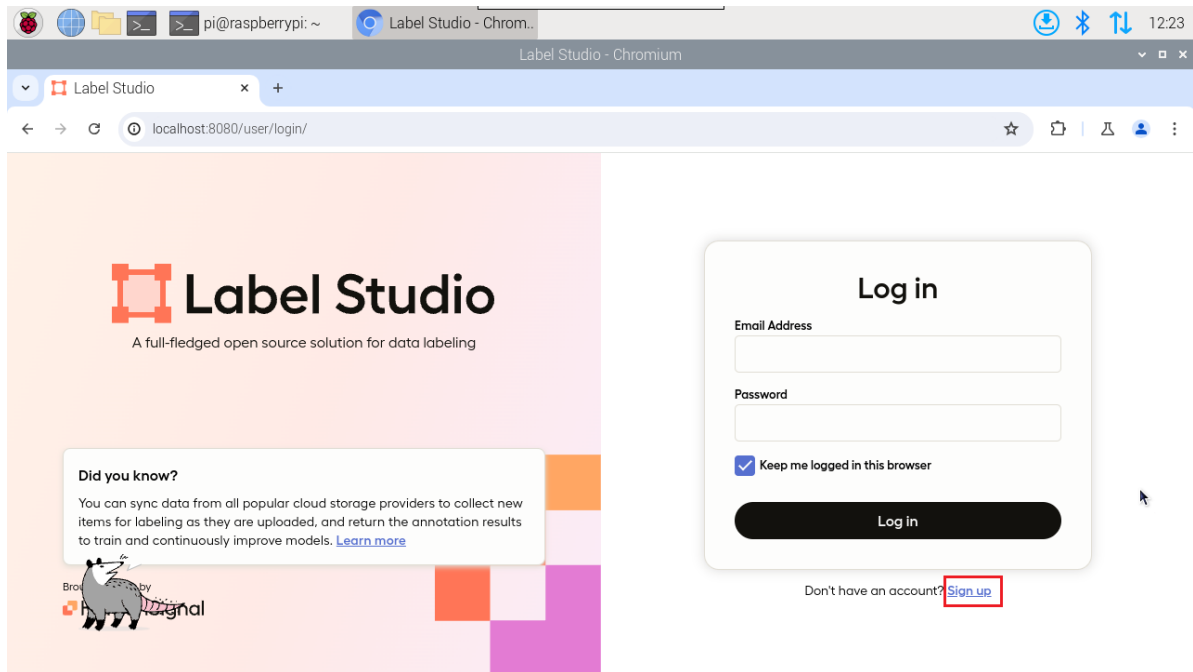
`http://BoardIP:8080/ # For example: http://192.168.2.99:8080/`



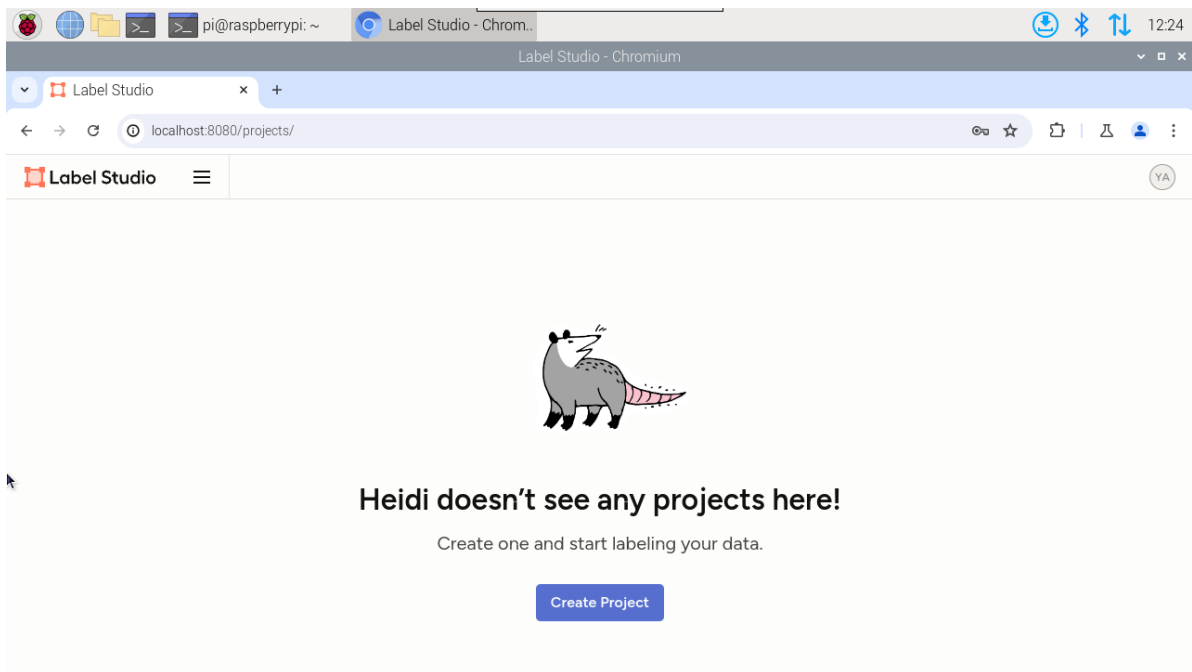
First-time Use

First-time use requires self-registration and login. Factory default account information (account information is fake):

Account: yahboom@163.com
Password: yahboom@163.com



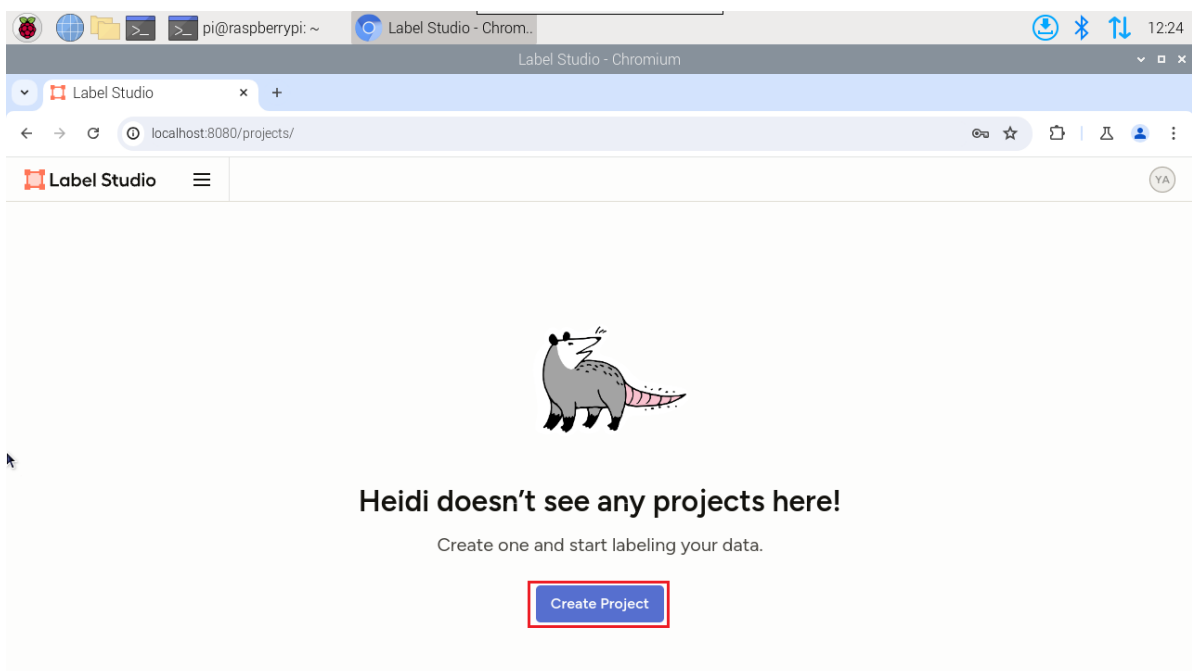
After registration is complete, the website will automatically log in:



3. Dataset Annotation

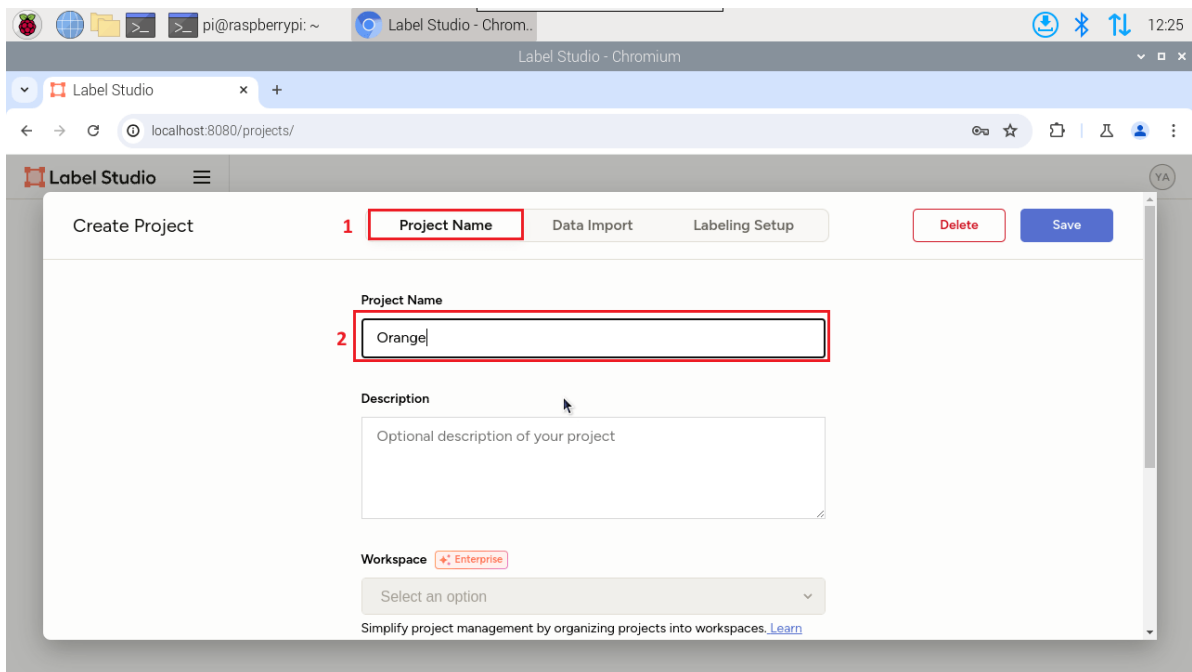
The demonstration dataset is on the board, so access Label Studio on the board.

3.1 Create Project



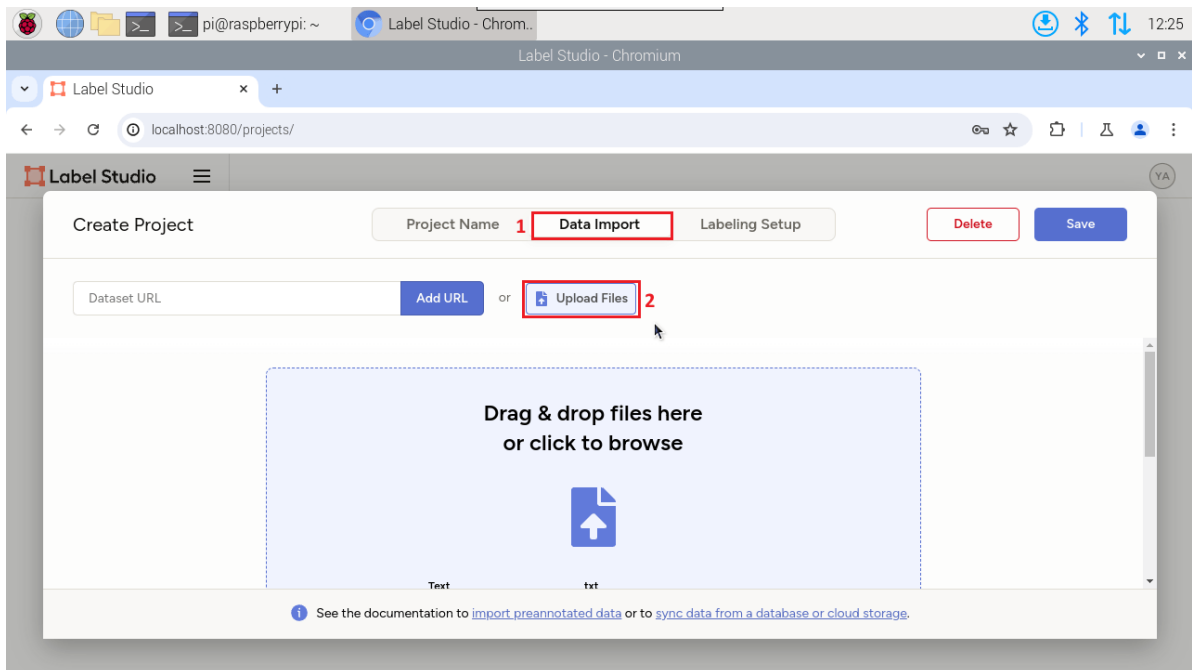
3.2 Project Name

The project name can be named arbitrarily. Name it according to your training set:



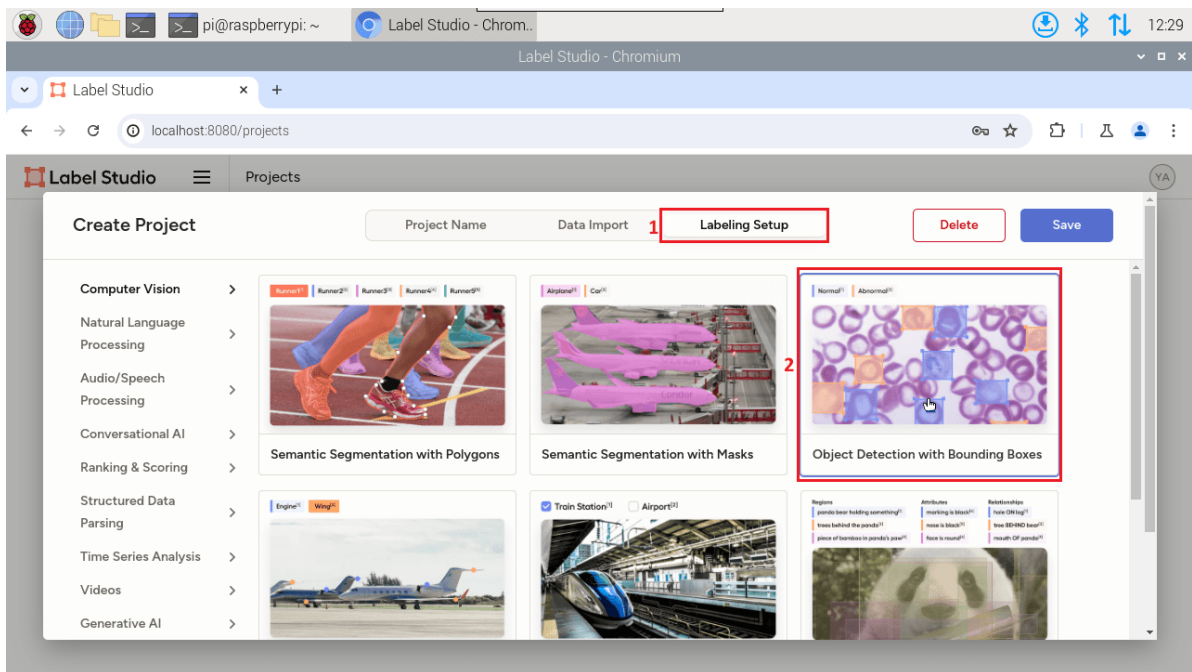
3.3 Import Training Set

Manually import the prepared dataset: Do not import too many images at once. It is recommended to select 50-100 images each time and import multiple times.

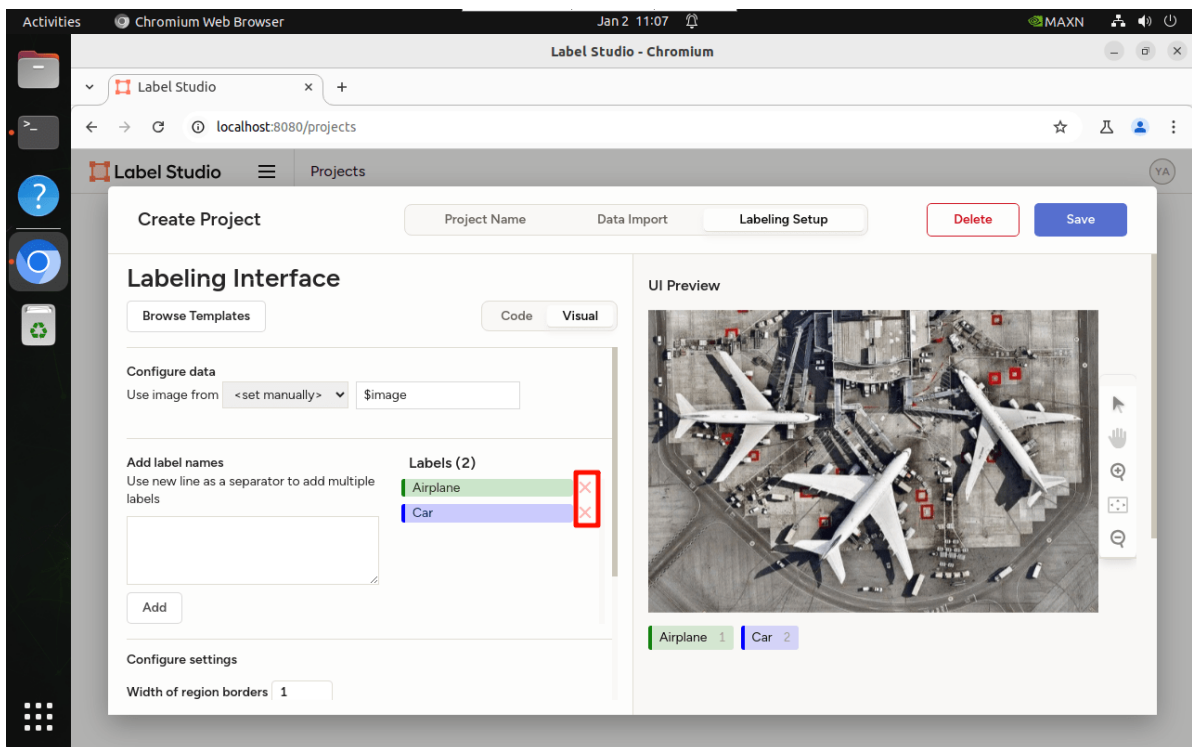


3.4 Label Settings

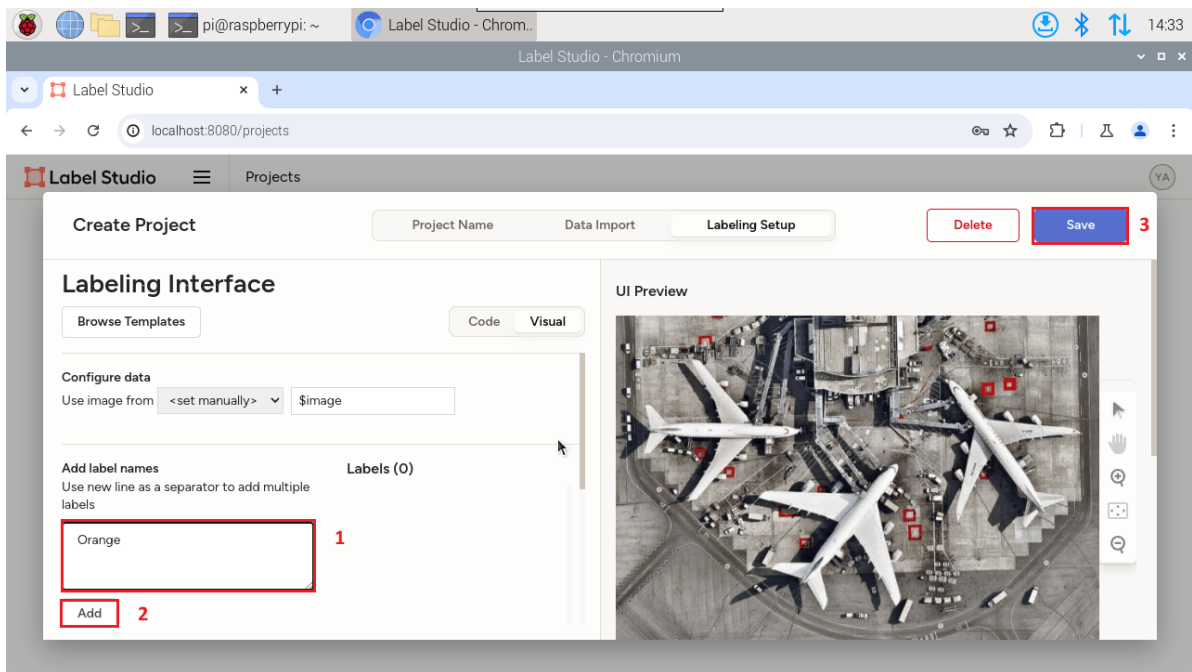
We are demonstrating orange recognition, so select "Object Detection".



Delete built-in labels:

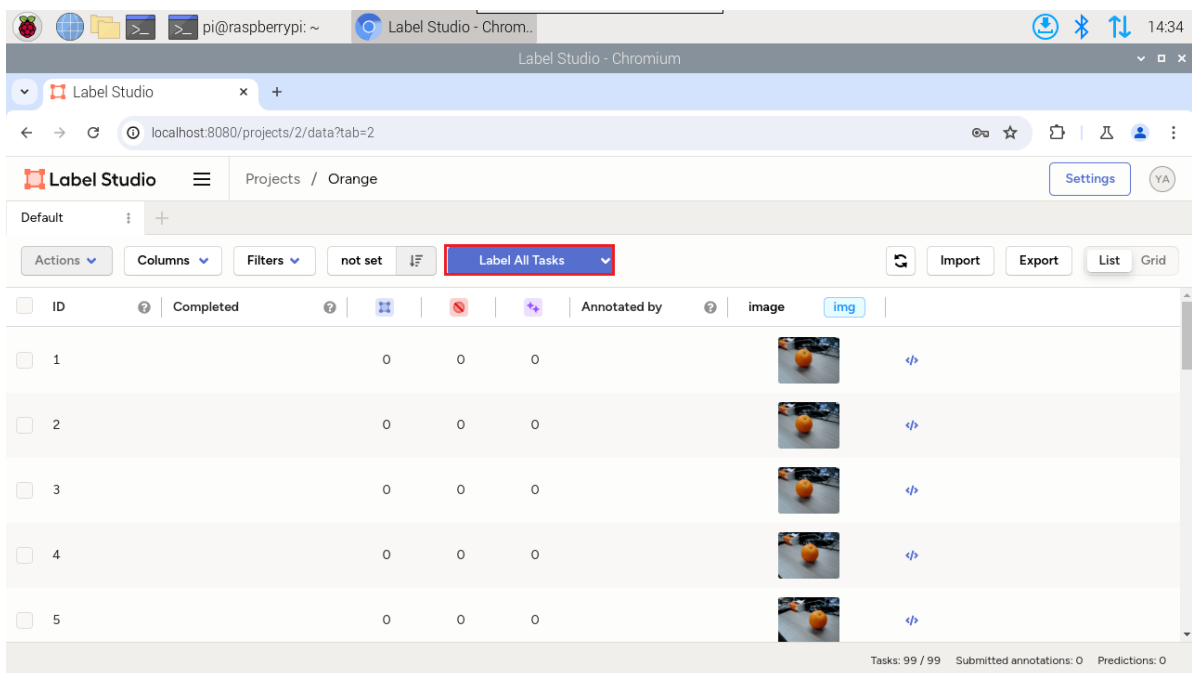


Add new labels:

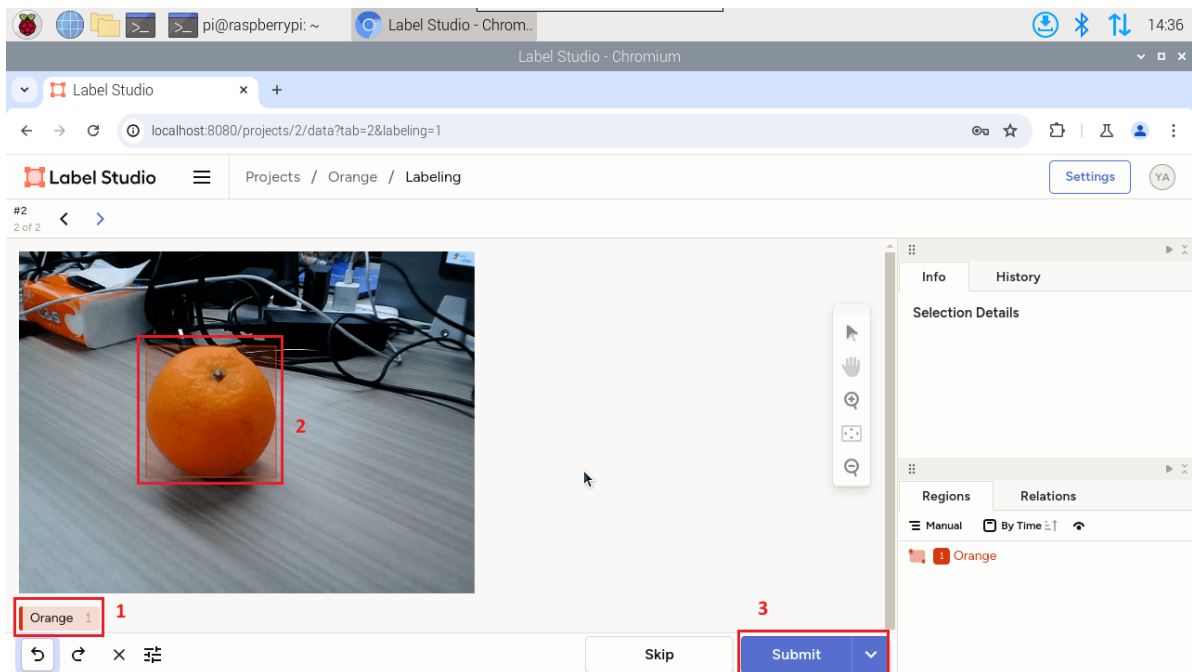


3.5 Dataset Annotation

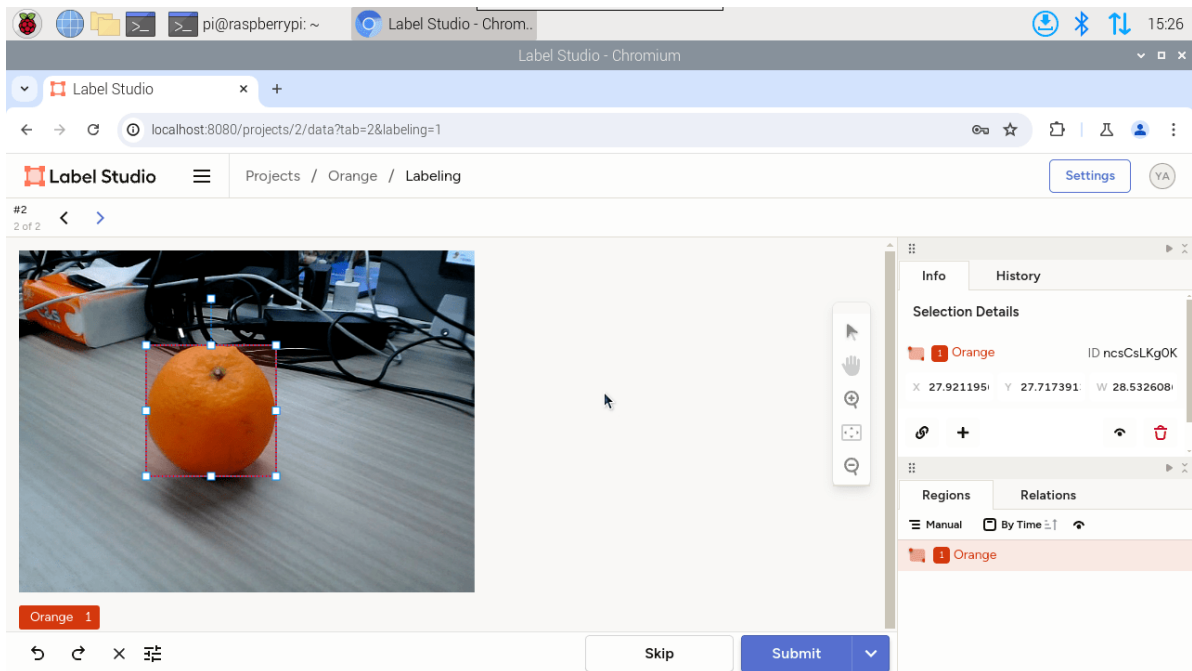
Click "Annotate All Tasks":



Annotation steps: First click the label "Orange" then use the mouse to select the corresponding orange in the image. After completion, click submit to proceed to the next annotation.

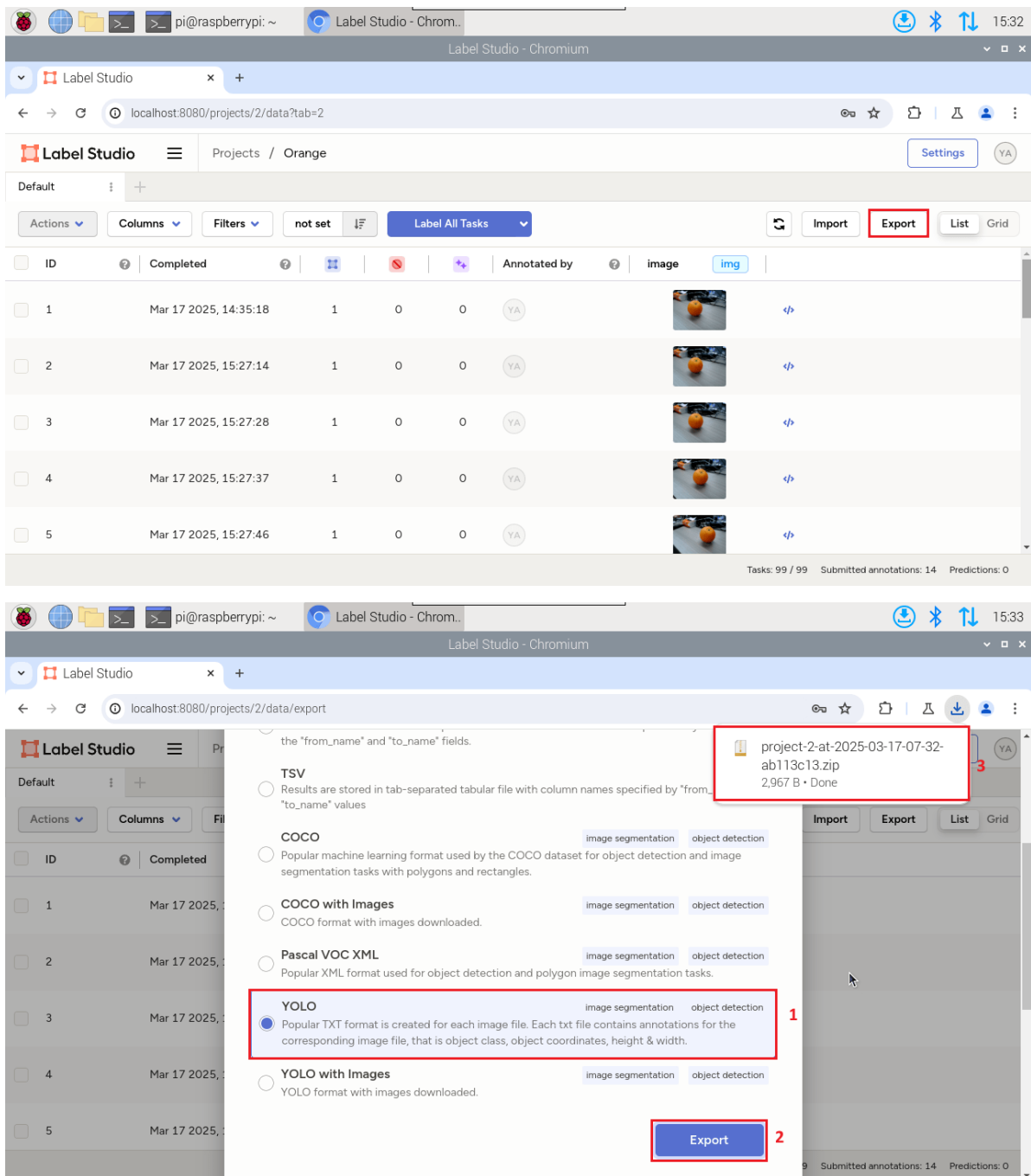


If the selected object is not a complete orange, you can click the selected box to readjust the position of each edge:



3.6 Export Dataset

Select YOLO format for export. The browser will automatically download a compressed package.



3.7 Dataset Directory Structure

Directory structure corresponding to the compressed package exported by Label Studio:

```

.
├── images
├── labels
├── classes.txt
└── notes.json

```

We need to separate the images and labels exported by Label Studio into train (training set) and val (validation set). The ratio is approximately 8:2. The images and labels in train and val need to correspond one-to-one by name.

```
.
├── classes.txt
├── notes.json
├── train
│   ├── images
│   └── labels
└── val
    ├── images
    └── labels
```

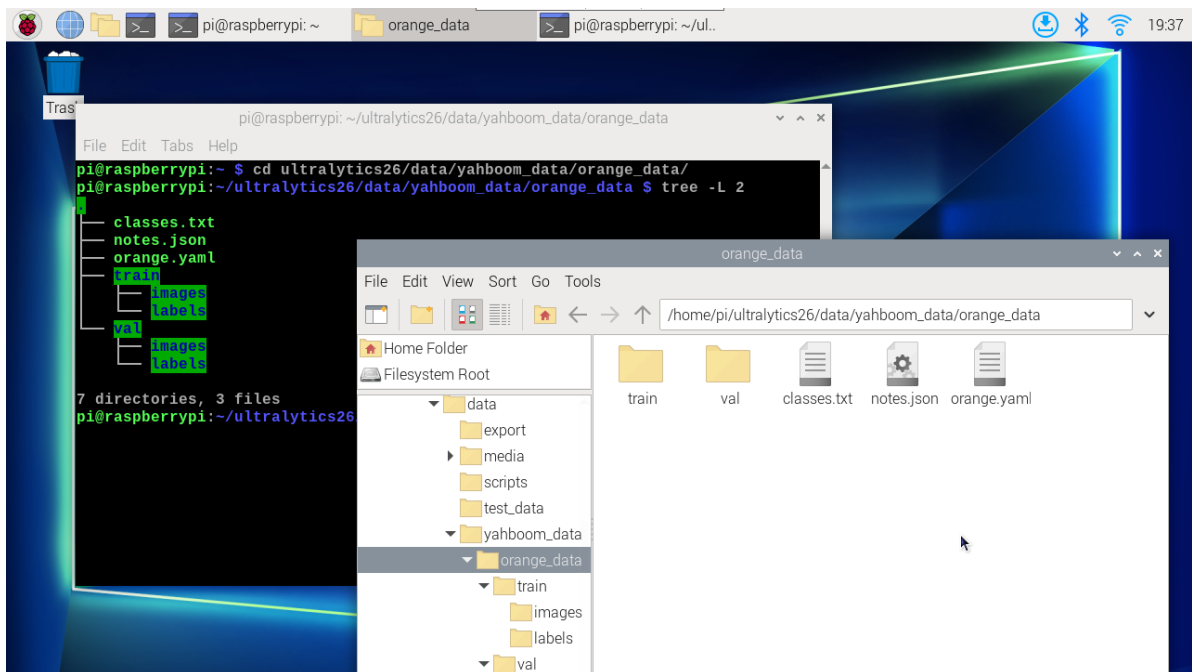
3.8 Dataset Configuration File

Contains paths to training and validation sets, as well as the number and names of classes.

```
path: /home/pi/ultralytics26/data/yahboom_data/orange_data
train: train/images
val: val/images

nc: 1

# Classes
names:
  0: orange
```



References

<https://github.com/HumanSignal/label-studio>